

TRABAJO PRACTICO INTEGRADOR

Gestion de Datos de Paises

en Python

Filtros | Ordenamientos | Estadísticas

INFORMACION DEL PROYECTO

Materia: Programacion 1

Juan Manuel Garcia Rezusta - Comision 22

Tomas Rebottaro - Comision 4

Cohorte Marzo 2026 - Fecha de entrega: Junio 2026

Tecnicatura Universitaria en Programacion - UTN TUPD

Indice de Contenidos

1. Marco Teorico

- > 1.1. Listas
- > 1.2. Diccionarios
- > 1.3. Funciones
- > 1.4. Estructuras condicionales y repetitivas
- > 1.5. Ordenamientos
- > 1.6. Estadisticas basicas
- > 1.7. Archivos CSV

2. Decisiones Tecnicas y Arquitectura

- > 2.1. Estructura del proyecto
- > 2.2. Flujo principal de operaciones
- > 2.3. Patron de retorno (booleano, mensaje)

3. Dificultades y Conclusiones

4. Bibliografia y Webgrafia

5. Links

1. Marco Teorico

1.1. Listas

En Python, una lista es una coleccion ordenada y mutable de elementos. Pueden contener objetos de cualquier tipo, incluyendo otros diccionarios o listas. En este proyecto, la estructura central es una **lista de diccionarios**: cada elemento representa un pais con sus cuatro campos. Las listas permiten agregar elementos con **append()**, recorrerlos con bucles **for**, y filtrarlos construyendo nuevas listas con condiciones.

```
países = []                # lista vacia al inicio
países.append(nuevo_pais)  # agrega un país
for item in países: ...    # recorre todos los países
```

1.2. Diccionarios

Un diccionario almacena pares clave-valor. El acceso a sus elementos se hace por clave y no por indice numerico. Cada pais se representa como un diccionario con las claves '**nombre**', '**poblacion**', '**superficie**' y '**continente**'. Esta estructura facilita el acceso semantico a los datos y la legibilidad del codigo.

```
pais = {"nombre": "argentina", "poblacion": 45376763,
        "superficie": 2780400, "continente": "america"}
```

1.3. Funciones

Una funcion es un bloque de codigo reutilizable que realiza una tarea especifica. El principio aplicado en este proyecto es '**una funcion, una responsabilidad**': cada funcion de **logica.py** hace exactamente una cosa. La separacion entre **logica.py** y **main.py** es en si misma una aplicacion de este principio a nivel de modulo.

```
def agregar_pais(nuevo, países): ...
def filtro_cont(países, continente): ...
def estadistica(países): ...
```

1.4. Condicionales y repetitivas

Las estructuras **if / elif / else** permiten tomar decisiones en funcion del estado de los datos. Los bucles **for** se usan para recorrer la lista en busquedas, filtros y estadísticas. El bucle **while True** en el menu principal mantiene el programa activo hasta que el usuario elige salir.

```
while True:
    mostrar_menu_principal()
    opcion = input('Elegi una opcion: ')
    if opcion == '7': break
```

1.5. Ordenamientos

Python provee la funcion incorporada **sorted()**, que devuelve una nueva lista ordenada sin modificar la original. Recibe un parametro **key** que indica el criterio de comparacion. Se usan funciones auxiliares (**saca_nombre**, **saca_poblacion**, **saca_superficie**) como criterio. El orden descendente se obtiene con **lista.reverse()**.

```
def saca_poblacion(p): return p['poblacion']
lista = sorted(paises, key=saca_poblacion)
lista.reverse() # para orden descendente
```

1.6. Estadísticas básicas

Las estadísticas se calculan con un unico recorrido de la lista: se acumulan totales, se comparan valores para encontrar maximo y minimo, y se usa un diccionario auxiliar **conteo_continentes** para contar paises por continente. Los promedios se calculan dividiendo los totales por la cantidad de paises al finalizar el recorrido.

```
pob_promedio += item['poblacion']
if item['poblacion'] > mas_poblado['poblacion']:
    mas_poblado = item
conteo_continentes[cont] += 1
```

1.7. Archivos CSV

CSV (Comma-Separated Values) es un formato de texto plano donde cada linea es un registro y los campos se separan por comas. Python incluye el modulo **csv** en su biblioteca estandar. **DictReader** lee cada fila como un diccionario y **DictWriter** escribe diccionarios como filas. El archivo **data.csv** actua como base de datos persistente: se carga al iniciar el programa y se guarda al salir.

```
lector = csv.DictReader(archivo)          # lee filas como dicts
paquete = csv.DictWriter(archivo, fieldnames=encabezados)
paquete.writeheader()
paquete.writerows(paises_formateados)
```

2. Decisiones Tecnicas y Arquitectura

2.1. Estructura del proyecto

El proyecto se divide en dos modulos con responsabilidades claramente separadas, mas el archivo de datos persistente:

Archivo	Responsabilidad
logica.py	Validaciones, operaciones sobre datos, filtros, estadisticas y acceso al CSV.
main.py	Menu interactivo, entrada del usuario y presentacion de resultados en consola.
data.csv	Base de datos persistente con los registros de paises.

2.2. Flujo principal de operaciones

Al iniciar, **main.py** llama a **logica.cargar_csv()** para leer el archivo y construir la lista de paises en memoria. Cada opcion del menu delega en la funcion correspondiente de **logica.py**. Al salir (opcion 7), se llama a **logica.guardar_csv()** para persistir los cambios.

1. Inicio -> cargar_csv() -> lista de paises en memoria
2. Menu principal -> el usuario elige una opcion (1-7)
3. main.py llama a la funcion correspondiente de logica.py
4. logica.py devuelve (True/False, resultado)
5. main.py evalua el booleano y muestra el resultado al usuario
6. Vuelve al paso 2 hasta que el usuario elige salir
7. Salida -> guardar_csv() -> datos persistidos en data.csv

2.3. Patron de retorno (booleano, mensaje)

Todas las funciones de **logica.py** devuelven una tupla de dos elementos: un booleano que indica exito o error, y un segundo valor que es el mensaje de error o el resultado (string o lista). Este patron permite que **main.py** maneje cualquier resultado de forma uniforme sin capturar excepciones, y hace que cada funcion sea facilmente testeable de forma independiente.

```
# En logica.py:
def agregar_pais(nuevo, paises):
    if ya_existe_pais(nuevo['nombre'], paises):
        return (False, 'El pais ya existe.')
    paises.append(linea_nueva)
    return (True, 'Pais agregado correctamente.')

# En main.py:
exito, mensaje = logica.agregar_pais(nuevo, paises)
print(mensaje)  # siempre hay un mensaje claro para el usuario
```

3. Dificultades y Conclusiones

Dificultades encontradas

Normalizacion de strings:

Una de las primeras dificultades fue decidir como representar los datos internamente. Se opto por normalizar nombre y continente a minusculas al cargar, y capitalizar solo al mostrar o guardar, lo que evito problemas de comparacion entre 'Argentina' y 'argentina'.

Validacion de tipos numericos:

El uso de `.isdigit()` no contempla el caso en que el dato llegue ya como entero. Se reemplazo por bloques `try/except` con `ValueError` y `TypeError`, mas robustos y que tambien validan que los valores sean positivos.

Separacion de responsabilidades:

La separacion entre logica y presentacion requirio planificacion previa: fue necesario definir desde el inicio que responsabilidad tenia cada modulo para evitar mezclar validaciones con mensajes al usuario.

Conclusiones

Este proyecto permitio consolidar el uso de listas y diccionarios como estructuras complementarias, y entender en la practica por que la modularizacion facilita el mantenimiento del codigo. La separacion en dos modulos resulto ser una decision acertada: permitio modificar la logica sin tocar la presentacion y viceversa.

El trabajo en equipo implico coordinar criterios de estilo y nomenclatura desde el inicio, lo cual fue un aprendizaje en si mismo sobre como se desarrolla software de forma colaborativa.

4. Bibliografia y Webgrafia

- [1] **Python Software Foundation.** *The Python Standard Library.*
<https://docs.python.org/3/library/>

- [2] **Python Software Foundation.** *csv - CSV File Reading and Writing.*
<https://docs.python.org/3/library/csv.html>

- [3] **Python Software Foundation.** *Built-in Functions: sorted().*
<https://docs.python.org/3/library/functions.html#sorted>

- [4] **Python Software Foundation.** *Data Structures: Dictionaries.*
<https://docs.python.org/3/tutorial/datastructures.html>

- [5] **Lutz, M. (2013).** *Learning Python, 5th ed..* O'Reilly Media

5. Links

- | | |
|----------------------------|---|
| Repositorio GitHub: | https://github.com/juanrezusta/TPI_programacion1 |
| Video demostrativo: | https://www.youtube.com/watch?v=6aJ65Ps_l68 |